

AI Resume Analyzer

NLP-powered resume screening and skill gap analysis system. Built with Python, FastAPI, spaCy, and sentence-transformers.

What It Does

- Extracts technical skills from resumes using NLP (Named Entity Recognition + regex pattern matching)
- Computes semantic similarity between resume and job description using transformer embeddings
- Identifies skill gaps — what's missing from your resume for a given role
- Ranks multiple resumes against a single job description (recruiter mode)
- Generates actionable improvement recommendations

Tech Stack

Layer	Technology
Language	Python 3.11+
Framework	FastAPI
NLP	spaCy (en_core_web_sm)
Embeddings	sentence-transformers (all-MiniLM-L6-v2)
ML Math	scikit-learn, numpy
PDF Parsing	PyMuPDF
Validation	Pydantic v2

No paid API keys needed. Everything runs locally. The embedding model (all-MiniLM-L6-v2) generates 384-dimensional vectors and runs fully on-device.

Architecture

```
CLIENT (Postman / Frontend)
  |
  v
FastAPI Application
  |
  +-----+-----+-----+
  | PDF Parser | Skill Extractor | Ranker |
  | (PyMuPDF) | (spaCy + regex) | (scoring)|
  +-----+-----+-----+
```

```
    |
    v
Embedding Engine (sentence-transformers / all-MiniLM-L6-v2)
384-dimension vectors – cosine similarity
    |
    v
Gap Analysis Engine
job_skills - resume_skills = missing_skills
+ semantic matching + priority scoring
```

API Endpoints

Method	Endpoint	Description
POST	/api/analyze	Analyze resume text vs job description
POST	/api/analyze-pdf	Upload PDF resume and analyze
POST	/api/extract-skills	Extract skills from any text
POST	/api/rank-resumes	Rank multiple resumes for one job
POST	/api/similarity	Compute semantic similarity between two texts
GET	/api/health	Health check

Local Setup

```
# Clone
git clone https://github.com/Ketansinghrajput/ai-resume-analyzer.git
cd ai-resume-analyzer

# Create virtual environment
python -m venv venv
venv\Scripts\activate      # Windows
# source venv/bin/activate  # Mac/Linux

# Install dependencies
pip install -r requirements.txt
python -m spacy download en_core_web_sm

# Run
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Visit <http://localhost:8000/docs> for interactive Swagger UI.

Sample Request

```
curl -X POST http://localhost:8000/api/analyze \
-H "Content-Type: application/json" \
-d '{
  "resume_text": "Java developer with Spring Boot, PostgreSQL, Docker, AWS,
  Microservices experience.",
  "job_description": "Looking for Java engineer with Spring Boot, REST APIs,
  PostgreSQL, Docker, and AWS."
}'
```

Response:

```
{
  "success": true,
  "data": {
    "overall_match": {
      "semantic_similarity": 0.8912,
      "skill_match_percentage": 100.0,
      "verdict": "Strong Match - You are well-qualified for this role"
    },
    "matching_skills": ["aws", "docker", "java", "postgresql", "spring boot"],
    "missing_skills": [],
    "recommendations": ["Your skills are a strong match. Focus on showcasing
    project experience."]
  }
}
```

How the ML Pipeline Works

Skill Extraction — Multi-strategy NLP pipeline:

- Pattern matching against a curated skill database (80+ tech skills)
- Word boundary regex to avoid false positives (java won't match javascript)
- spaCy tokenization for context-aware parsing
- Frequency scoring to measure skill emphasis

Semantic Similarity — Transformer embeddings:

- Text converted to 384-dimensional vectors using all-MiniLM-L6-v2
- Cosine similarity measures angle between vectors
- Similar meanings cluster together regardless of exact wording

Gap Analysis — Set operations + semantic ranking:

- $missing = job_skills - resume_skills$
- Missing skills ranked by relevance to job using embedding similarity
- Weighted verdict combining semantic score + skill match percentage

Project Structure

```
ai-resume-analyzer/  
├── app/  
│   ├── api/  
│   │   ├── routes.py  
│   │   └── schemas.py  
│   ├── core/  
│   ├── data/  
│   │   └── tech_skills.json  
│   ├── services/  
│   │   ├── skill_extractor.py  
│   │   ├── embedding_service.py  
│   │   ├── gap_analyzer.py  
│   │   ├── pdf_parser.py  
│   │   └── resume_ranker.py  
│   └── main.py  
├── tests/  
├── requirements.txt  
└── run.py
```

Author

Ketansingh Rajput

[GitHub](#) · [LinkedIn](#)